Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07_Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет МДК 03.01 Внедрение и поддержка компьютерных систем

Тема: Проектирование безопасных релизных процессов, паттернов миграции данных и эксплуатационной документации ИС «Умный кампус»

ВАРИАНТ № 10 Спортивный комплекс (CAMPUS-110: изменение формата TimeSlot → TimeRange)

Студент (ка)/учащийся: Леонов Тимофей Андреевич /_____ /
Ф. И. О. подпись

Группа: И14-23-9

Руководитель: Чудинова А.А /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «22» октября 2026г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА.....3
2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД (OPENTELEMETRY).....5
3 CHANGE REQUEST НА РЕЛИЗ V1.9.0.....7
4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN (BRANCH PROTECTION).....10
5 МИГРАЦИЯ БД ПО ПАТТЕРНУ EXPAND AND CONTRACT.....12
6 PRODUCTION SANITY-ТЕСТ.....16
7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ (ADR, CHANGELOG).....18
ЗАКЛЮЧЕНИЕ.....20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....21

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА

Вариант № 10. Модуль ИС «Умный кампус» — **«Спортивный комплекс»** (бронирование залов, бассейна, кортов). Задача Jira — **CAMPUS-110**. Суть миграции базы данных — изменение формата поля TimeSlot со строки (string) на структуру TimeRange.

Исходное состояние (версия 1.8.3): интервал брони хранится в колонке sports.bookings.time_slot varchar(32) в виде строки 2026-11-05 18:00-19:30. Это приводит к двойному бронированию одной площадки (пересечение проверяется строками в коде, неатомарно), к путанице часовых поясов между мобильным приложением, сайтом и турникетами, к наличию двух форматов строки в базе и к невозможности индексировать поиск свободных окон.

Целевое состояние: в C# — value object TimeRange(Start, End) (полуоткрытый интервал [Start, End) в UTC), в PostgreSQL — встроенный диапазонный тип tstzrange с ограничением исключения EXCLUDE USING gist, которое физически запрещает пересекающиеся активные брони одной площадки.

Все исходные файлы размещены в публичном репозитории в папке homework-5/: https://github.com/kotokotru/smart-campus. Сервис реализован на ASP.NET Core 8, СУБД — PostgreSQL 16, тесты — xUnit, CI — GitHub Actions.

Блок задания	**Файл в репозитории**
1. Цифровой след	src/Campus.Sports.Api/Controllers/ SportsBookingsController.cs, Telemetry/CampusTelemetry.cs
2. Change Management	docs/change-requests/CR-2026-110-v1.9.0.md, docs/branch-protection.md
3. Expand and Contract	db/migrations/01_expand.sql, 02_migrate.sql, 03_contract.sql, Data/NpgsqlBookingRepository.cs
4. Sanity и Docs-as-Code	tests/Campus.Sports.Tests/ ProductionSanityTests.cs, docs/adr/0009-timeslot-to-timerange.md, CHANGELOG.md
Отчёт	REPORT.md, report/ (PDF)

Структура папки homework-5/ в репозитории показана на рисунке 1.

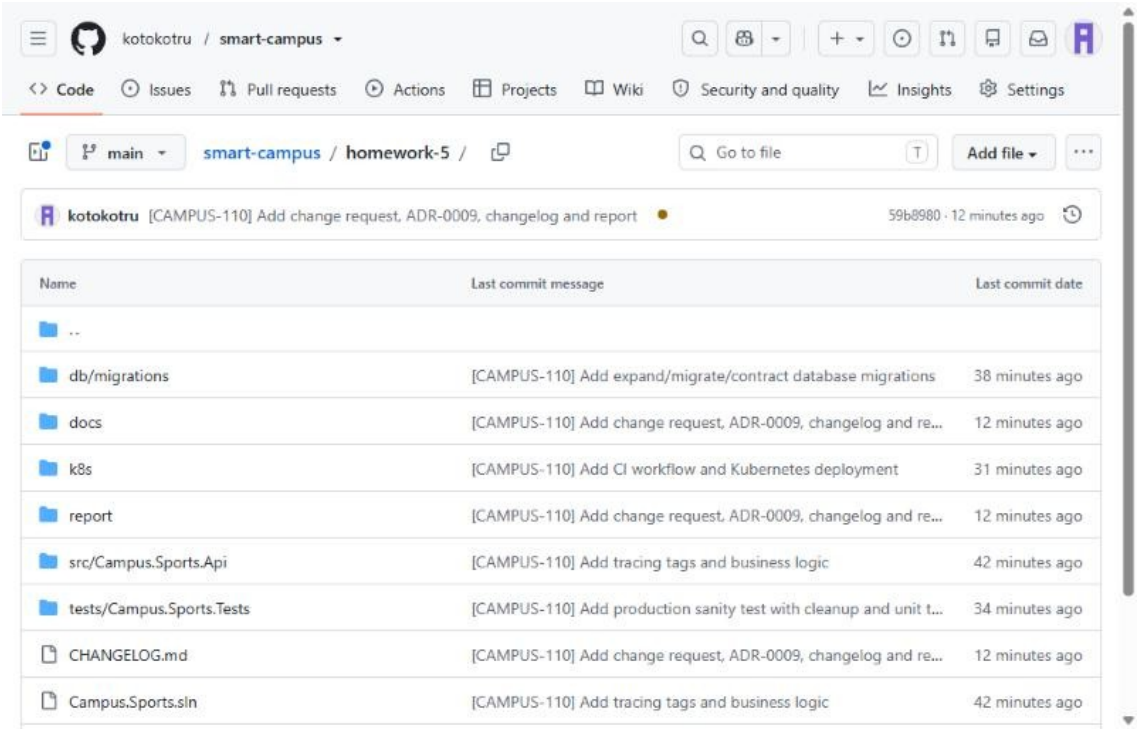


Рисунок 1 — Папка homework-5 в публичном репозитории kotokotru/smart-campus

2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД (OPENTELEMETRY)

В контроллер SportsBookingsController добавлен проброс цифрового следа OpenTelemetry. Теги устанавливаются в текущий span HTTP-запроса Activity.Current, который создаёт ASP.NET Core instrumentation, и в дочерний доменный span SportsBooking.Create из собственного ActivitySource("Campus.Sports") [1, 2] (рисунок 2).

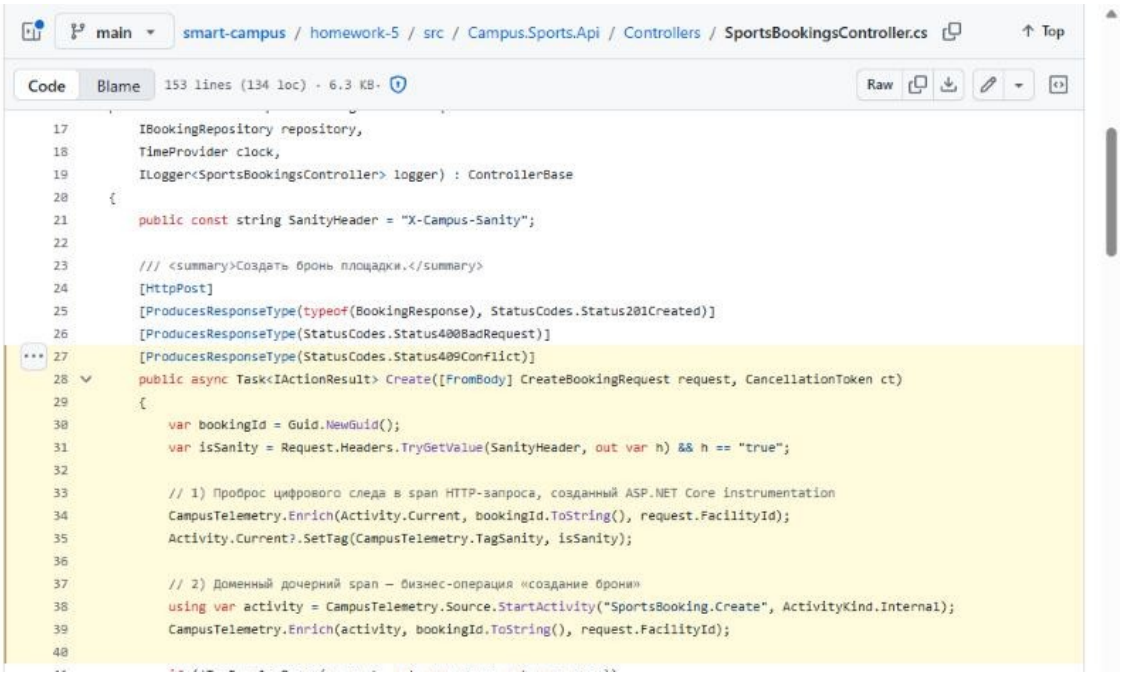


Рисунок 2 — Проброс тегов в Activity.Current и доменный span в методе Create (GitHub)

Листинг 1 — Установка обязательных тегов (CampusTelemetry.Enrich)

```
public static void Enrich(Activity? activity, string? bookingId = null,
                          string? facilityId = null)
{
   if (activity is null) return;
   activity.SetTag("jira.issue", "CAMPUS-110");
   activity.SetTag("campus.db.migration_phase", "expand");
   if (bookingId  is not null) activity.SetTag("campus.booking_id",  bookingId);
   if (facilityId is not null) activity.SetTag("campus.facility_id", facilityId);
}
```

Обязательные теги: jira.issue = CAMPUS-110 и доменный идентификатор сущности campus.booking_id. Дополнительно — campus.facility_id, campus.time_range.start/end и campus.request.format: по последнему тегу видно, сколько клиентов ещё присылают старый формат, и можно принять решение о фазе Contract. В Program.cs источник

зарегистрирован через AddOpenTelemetry().WithTracing(t => t.AddSource("Campus.Sports").AddAspNetCoreInstrumentation().AddOtlpExporter()), в ресурс добавлены service.version = 1.9.0 и SHA коммита.

Корректность установки тегов проверяет тест TracingTagsTests: через ActivityListener он перехватывает завершённые span’ы и проверяет, что и доменный, и родительский HTTP-span содержат jira.issue = CAMPUS-110 и тот же campus.booking_id, что вернул API.

Изменения зафиксированы Git-коммитом с сообщением [CAMPUS-110] Add tracing tags and business logic (8fbed04, https://github.com/kotokotru/smart-campus/commit/8fbed045e8c0b23976d226309973a792eae44bc6). Формат [CAMPUS-NNN] проверяется в CI (job commit-lint). В итоге формируется сквозная цепочка CALS-цифрового следа: задача Jira → коммит → CI → образ 1.9.0 → аннотация kubernetes.io/change-cause → span с jira.issue → запись в public.schema_migrations. История коммитов репозитория приведена на рисунке 3.

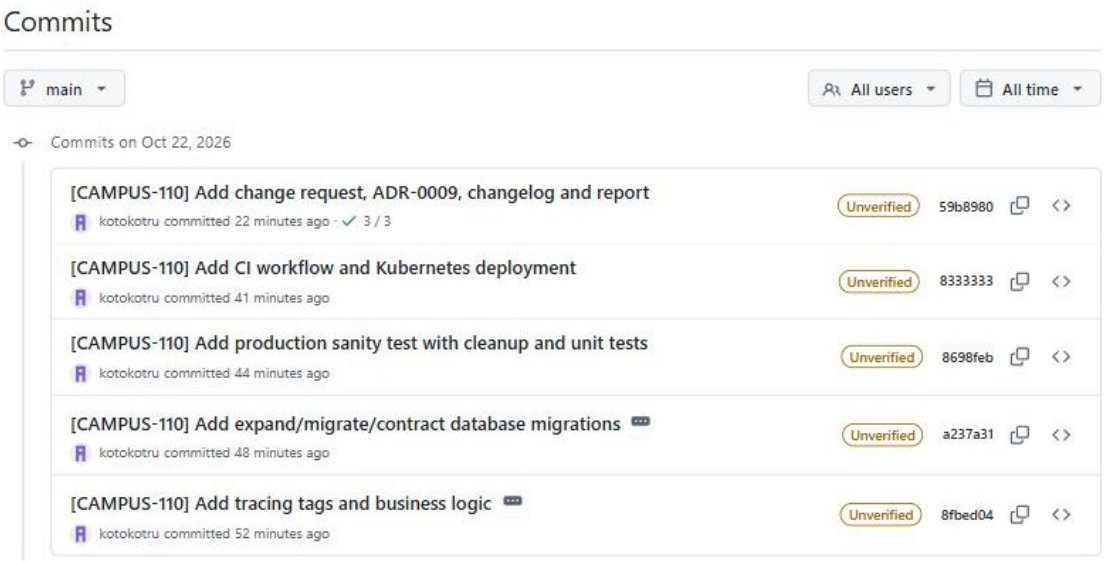


Рисунок 3 — История коммитов: каждый коммит привязан к задаче [CAMPUS-110]

3 CHANGE REQUEST НА РЕЛИЗ V1.9.0

Карточка Change Request оформлена в файле docs/change-requests/CR-2026-110-v1.9.0.md [6, 7]. Основные поля приведены в таблице 1, вид документа в репозитории — на рисунке 4.

Таблица 1 — Карточка CR-2026-110

Поле	**Значение**
Номер / задача	CR-2026-110 / CAMPUS-110
Сервис	campus-sports-api, v1.8.3 → v1.9.0
Тип, риск	Normal Change (CAB), риск средний; ожидаемый простой — 0 мин
Описание	Ввод TimeRange/tstzrange вместо строки TimeSlot; фазы Expand и Migrate; API принимает и возвращает оба формата; теги OpenTelemetry; Sanity-тест
Бизнес-цель	Исключить двойное бронирование площадок (37 обращений за сентябрь → 0), единое время брони во всех клиентах, основа для поиска свободных окон (CAMPUS-118)
Окно обслуживания	Среда 28.10.2026, 03:00–05:00 МСК
Согласующие	Тимлид платформы, DBA, Product Owner модуля, руководитель спорткомплекса

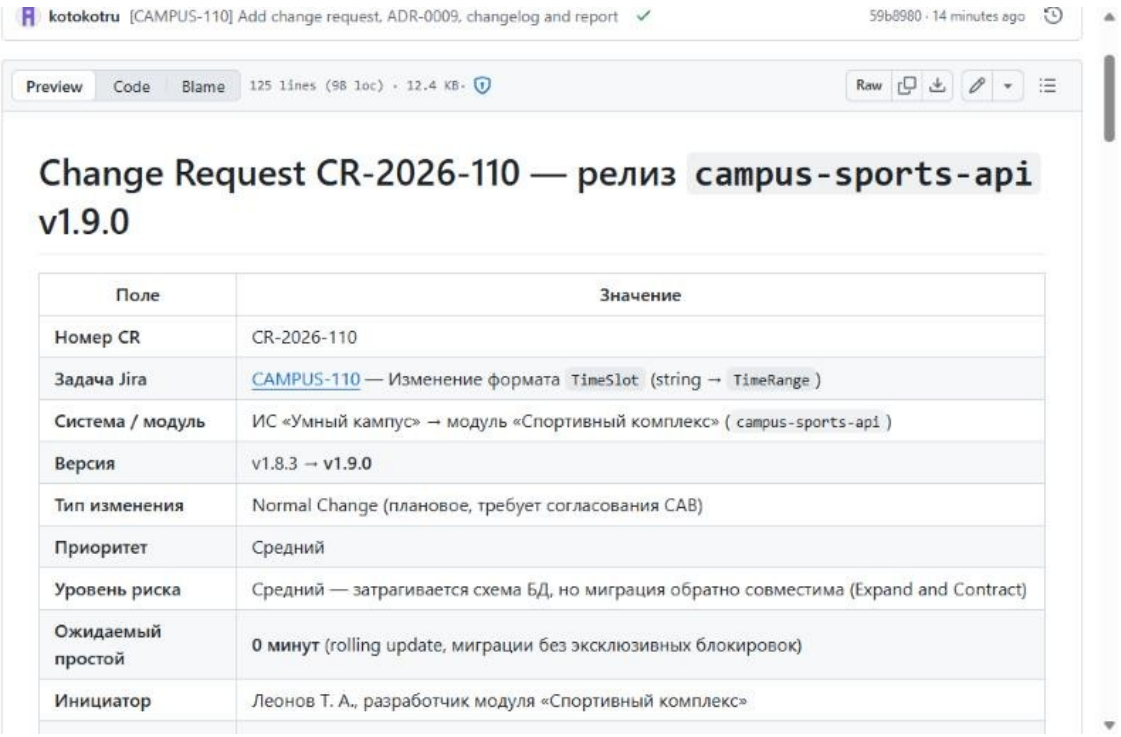


Рисунок 4 — Карточка CR-2026-110 в репозитории (docs/change-requests)

Обоснование окна обслуживания. По данным мониторинга за 8 недель минимальный трафик сервиса — 03:00–05:00 (≈ 2 RPS против 180 RPS в пике 17:00–21:00). Спорткомплекс закрыт с 23:00 до 07:00, турникеты

не работают. Ночной бэкап БД завершается в 02:30, поэтому к началу работ есть свежая резервная копия. До открытия в 07:00 остаётся два часа на откат. Среда выбрана потому, что в понедельник в 08:00 открывается запись на неделю (всплеск броней), а релиз в пятницу оставил бы проблему на выходные без дежурной команды.

План проведения работ:

1. 03:00 — фиксация текущей ревизии: kubectl rollout history deployment/campus-sports-api -n campus-sports.
2. 03:05 — фаза Expand: psql -v ON_ERROR_STOP=1 -f 01_expand.sql (DBA).
3. 03:10 — выкат: kubectl set image deployment/campus-sports-api api=…:1.9.0 и аннотация change-cause.
4. 03:12 — контроль: kubectl rollout status … --timeout=5m (rolling update, maxUnavailable: 0).
5. 03:20 — Production Sanity-тест: dotnet test --filter "Category=Sanity".
6. 03:25 — фаза Migrate: 02_migrate.sql, проверка «NULL = 0, errors = 0, mismatches = 0».
7. 03:35 — проверка трейсов по jira.issue=CAMPUS-110; до 04:30 — наблюдение (5xx, p95, pg_locks).
8. 04:30 — закрытие CR, запись в CHANGELOG, оповещение в канале релизов.

План отката. Критерии: доля 5xx > 1 % в течение 5 минут, p95 > 800 мс, падение Sanity-теста или rollout status не завершился за 5 минут. Действия (целевое время ≤ 5 мин) приведены на рисунке 5.



`rollout status` не завершился за 5 минут; ошибки в `02_migrate.sql`, которые нельзя устранить за 15 минут.

Решение об откате принимает релиз-инженер совместно с дежурным SRE. Целевое время отката — **≤ 5 минут.**

Шаг	Команда	Комментарий
1. Откат приложения	`kubectl rollout undo deployment/campus-sports-api -n campus-sports`	возврат на предыдущую ревизию (v1.8.3)
2. Контроль	`kubectl rollout status deployment/campus-sports-api -n campus-sports --timeout=5m`	все поды v1.8.3 `Ready`
3. Проверка	`kubectl rollout history deployment/campus-sports-api -n campus-sports` + smoke-запрос `GET /health/ready`	—
4. БД — ничего не делать	Схема после Expand обратно совместима: v1.8.3 не знает о `time_range`, а триггер продолжает заполнять колонку. Данные в `time_slot` актуальны (dual-write).	откат БД не требуется
5. БД — полный откат (по решению DBA)	`psql -f db/migrations/01_expand_rollback.sql`	только если проблема в самой схеме (например, триггер); удаляет `time_range`

Рисунок 5 — План отката в CR: kubectl rollout undo, контроль и действия с БД

Откат БД не требуется: после фазы Expand схема обратно совместима — версия 1.8.3 не знает о колонке time_range, а time_slot продолжает заполняться (dual-write и триггер). При проблеме в самой схеме DBA выполняет 01_expand_rollback.sql. Точка невозврата наступает только в релизе v1.10.0 при выполнении 03_contract.sql, который оформляется отдельным CR.

4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN (BRANCH PROTECTION)

Конфигурация защиты ветки main описана в docs/branch-protection.md и продублирована в виде JSON для применения через GitHub API (gh api -X PUT repos/{owner}/{repo}/branches/main/protection) [3].

Таблица 2 — Branch Protection Rules для ветки main

Настройка	**Значение**
Require a pull request before merging	Да — прямой push в main запрещён
Required approvals	2 одобрения
Require review from Code Owners	Да — изменения db/migrations/ одобряет DBA
Dismiss stale approvals / Require approval of the most recent push	Да / Да
Require status checks to pass	commit-lint, build-and-test, db-migrations
Require branches to be up to date	Да
Require conversation resolution	Да
Require signed commits / linear history	Да / Да
Do not allow bypassing (включая админов)	Да
Allow force pushes / deletions	Нет / Нет

Обязательные проверки CI определены в .github/workflows/homework-5-ci.yml: commit-lint проверяет, что каждый коммит и заголовок PR начинаются с [CAMPUS-NNN]; build-and-test собирает решение и запускает все xUnit-тесты; db-migrations поднимает PostgreSQL 16 и последовательно применяет миграции 00 → 01 → 02 (дважды, проверка идемпотентности) → 03, а также проверяет откат фазы 1 на чистой базе. Для тегов v* действует защита тегов, а деплой в окружение production требует ручного одобрения тимлида и SRE.

Все три обязательные проверки выполнены успешно на последнем коммите ветки main (рисунок 6).

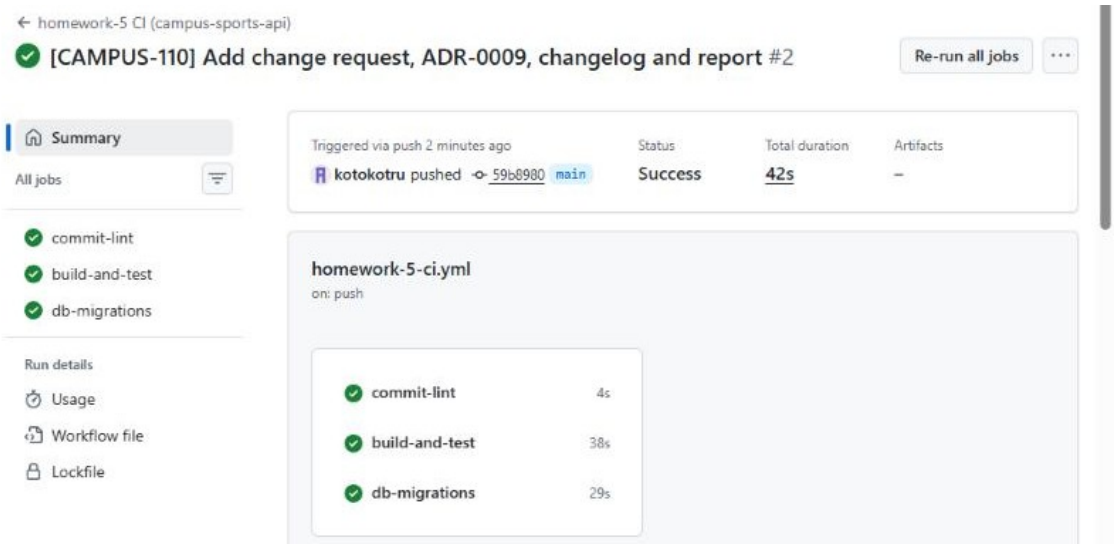


Рисунок 6 — GitHub Actions: проверки commit-lint, build-and-test, db-migrations пройдены

5 МИГРАЦИЯ БД ПО ПАТТЕРНУ EXPAND AND CONTRACT

Миграция разделена на три фазы и два релиза, чтобы в каждый момент времени со схемой могли работать и старые (1.8), и новые (1.9) экземпляры приложения — без простоя [4, 5, 8].

Фаза 1 — Expand (01_expand.sql, v1.9.0). Добавляется колонка time_range tstzrange без значения по умолчанию — это изменение только каталога, выполняется за миллисекунды. CHECK-ограничение корректности интервала создаётся NOT VALID (без сканирования таблицы). Создаются функции parse_legacy_time_slot / format_legacy_time_slot и триггер синхронизации, который во время rolling update заполняет новую колонку для записей от подов 1.8. Индекс GiST по (facility_id, time_range) строится CONCURRENTLY. Установлен lock_timeout = 3s (рисунок 7).

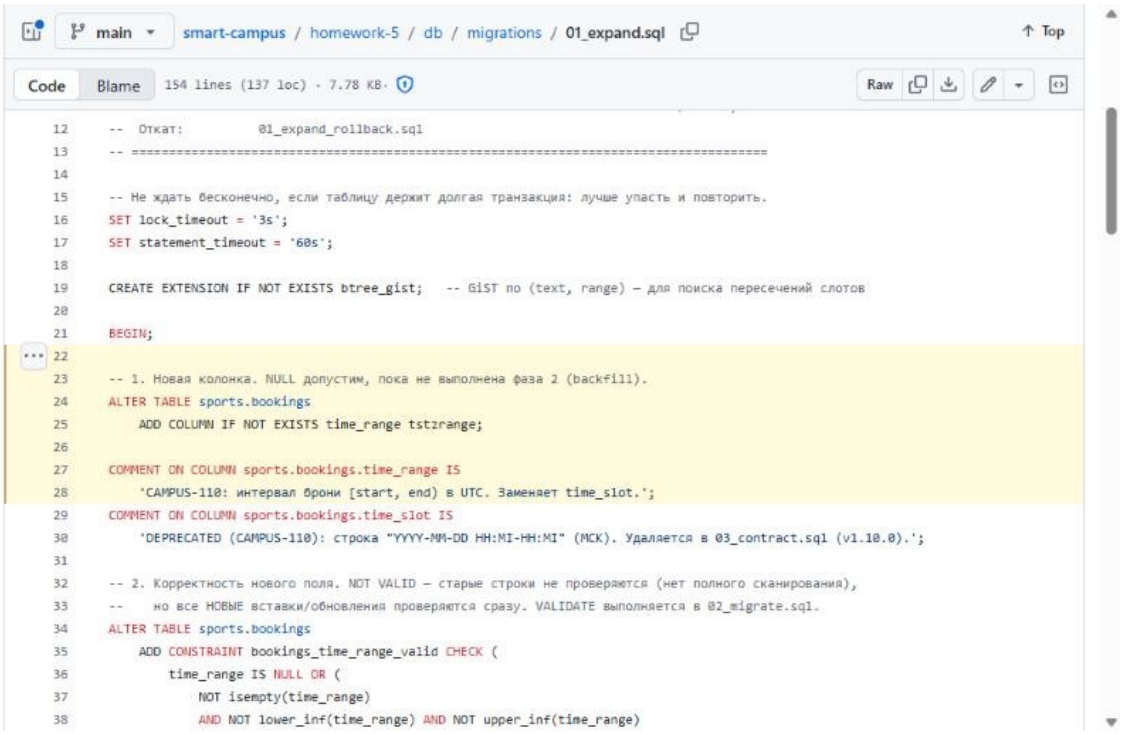


Рисунок 7 — Фаза Expand: добавление колонки time_range и CHECK NOT VALID (01_expand.sql)

C#-код одновременно пишет данные в старую и новую структуру (dual-write) в одной команде INSERT; запись в старую колонку управляется флагом Migration:WriteLegacyTimeSlot (рисунок 8).

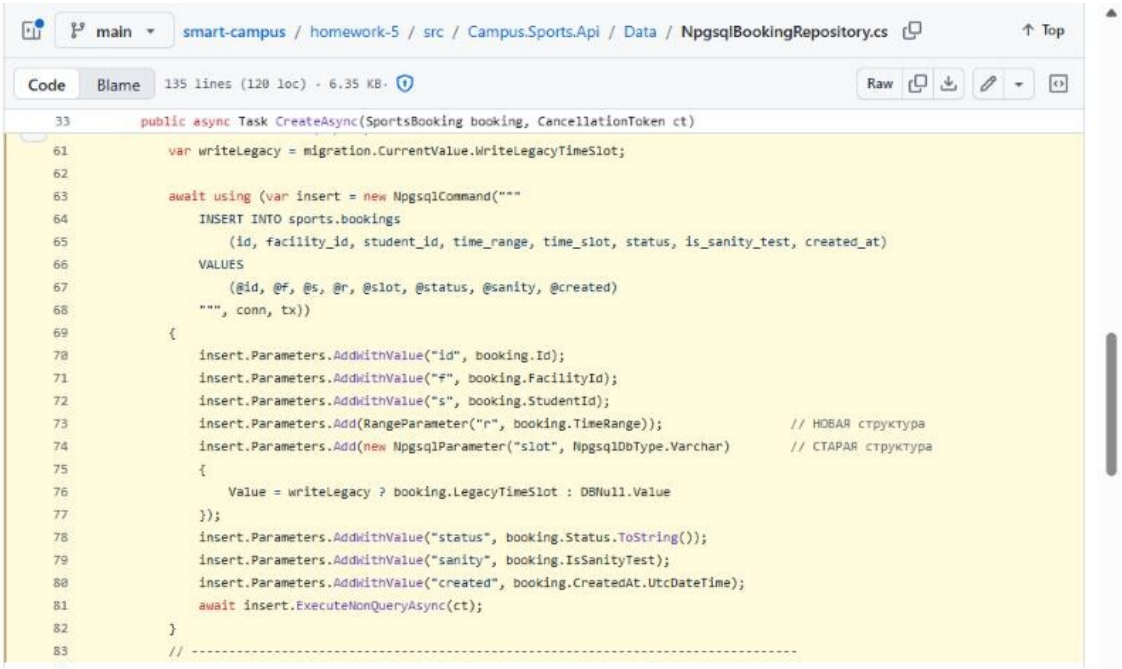


Рисунок 8 — Dual-write: одна команда INSERT пишет новую (time_range) и старую (time_slot) структуры

Фаза 2 — Migrate (02_migrate.sql, v1.9.0). Процедура sports.backfill_time_range переносит исторические данные пакетами по 5 000 строк; каждый пакет — отдельная короткая транзакция (COMMIT внутри процедуры), строки выбираются FOR UPDATE SKIP LOCKED, между пакетами — пауза. Нераспознанные строки записываются в журнал sports.bookings_migration_errors. После переноса выполняется проверка (нет NULL, нет ошибок, нет расхождений), затем создаётся CHECK (time_range IS NOT NULL) NOT VALID и выполняется VALIDATE CONSTRAINT — с блокировкой, не мешающей записи. Скрипт идемпотентен. Журнал выполнения в CI — на рисунке 9.

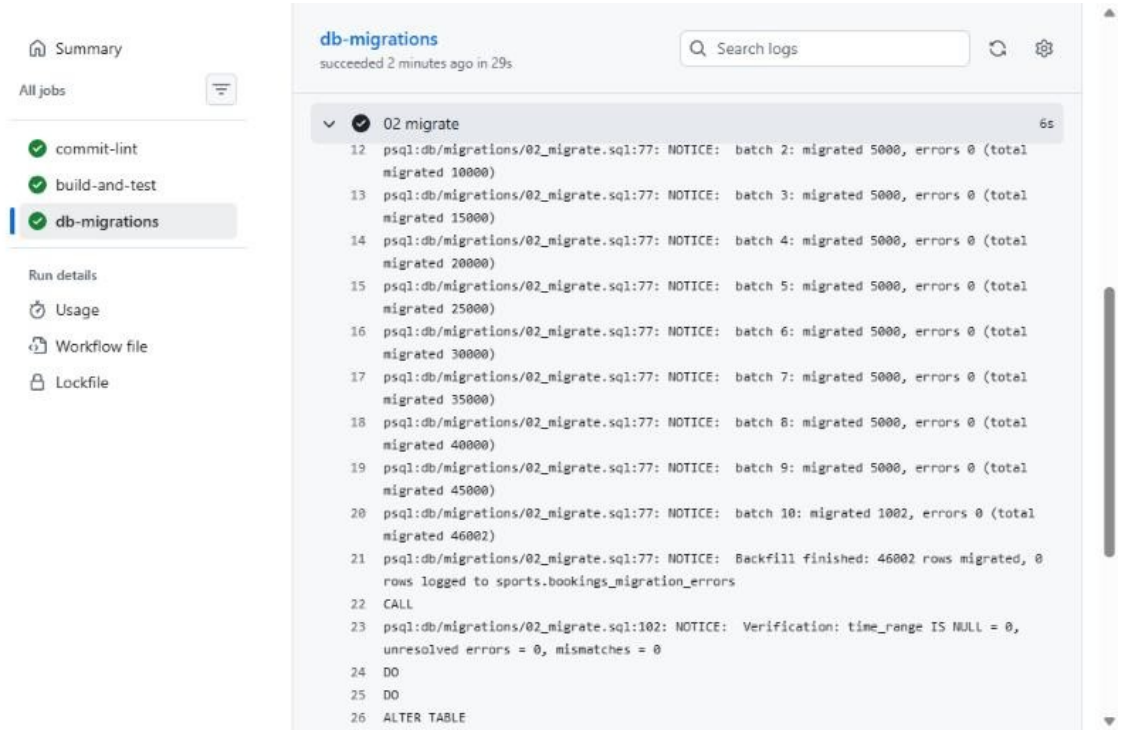


Рисунок 9 — Выполнение 02_migrate.sql в CI: 10 пакетов, 46 002 строки, проверка без ошибок

Фаза 3 — Contract (03_contract.sql, v1.10.0). Выполняется после перевода всех клиентов и 14 дней без запросов в старом формате. Скрипт проверяет предусловия, удаляет триггер, выполняет SET NOT NULL (без сканирования благодаря валидному CHECK), удаляет колонку time_slot и служебные функции, добавляет EXCLUDE USING gist (facility_id WITH =, time_range WITH &&).

Проверка. Все скрипты выполнены на PostgreSQL 16 на демо-наборе из 46 002 броней (результаты в таблице 3); тот же сценарий автоматически повторяет CI.

Таблица 3 — Результаты прогона миграций

Шаг	**Результат**
01_expand.sql	0,35 с; запись от «старого» клиента без time_range — триггер заполнил интервал
02_migrate.sql (с ошибочной строкой)	46 002 строки перенесены за 10 пакетов; строка «завтра вечером» попала в журнал ошибок, скрипт остановился до VALIDATE
02_migrate.sql (после исправления)	NULL = 0, errors = 0, mismatches = 0; ограничения VALID; повторный запуск — без изменений
03_contract.sql	time_slot удалена, time_range NOT NULL; пересекающаяся бронь → ошибка exclusion constraint bookings_no_overlap
01_expand_rollback.sql	схема возвращена к версии 1.8, данные (46 002

	строки) сохранены

6 PRODUCTION SANITY-ТЕСТ

Тест CreateBooking_InTimeRangeFormat_ThenCleanup класса ProductionSanityTests (xUnit, категория Sanity) выполняет точечную проверку боевого сценария сразу после релиза — создание тестовой брони:

1. создаётся бронь служебной площадки sanity-court-00 в новом формате start/end на 15 минут в ночное время случайного дня через год и более — чтобы не мешать реальным пользователям; заголовок X-Campus-Sanity: true помечает её как тестовую;
2. проверяется 201 Created, точное совпадение start/end и заполнение старого поля timeSlot (dual-write);
3. бронь читается через GET и сравнивается с ответом создания;
4. повторная бронь того же слота должна вернуть 409 Conflict (проверка пересечения интервалов);
5. **Cleanup** в DisposeAsync выполняется всегда, даже при падении проверок: DELETE (sanity-бронь удаляется физически) и контроль, что GET возвращает 404 (рисунок 10).

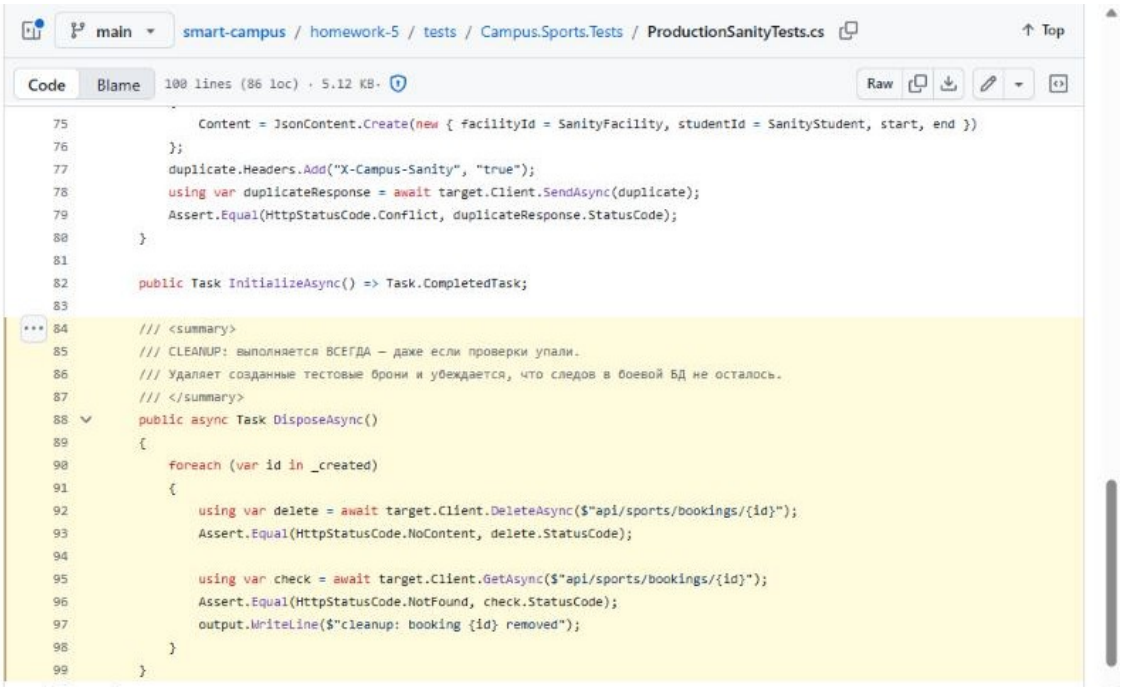


Рисунок 10 — Шаг Cleanup в DisposeAsync: удаление тестовой брони и проверка 404

Запуск после релиза — с переменной окружения SANITY_BASE_URL (адрес боевого стенда) и фильтром --filter Category=Sanity. Если переменная

не задана, фикстура SanityTarget поднимает сервис in-process через WebApplicationFactory, поэтому тот же сценарий выполняется в CI на каждом PR.

Результат прогона dotnet test на рабочем компьютере — 12 тестов из 12 пройдено, в том числе Sanity-тест (рисунок 11); тот же результат получен в GitHub Actions на .NET 8 (рисунок 12).

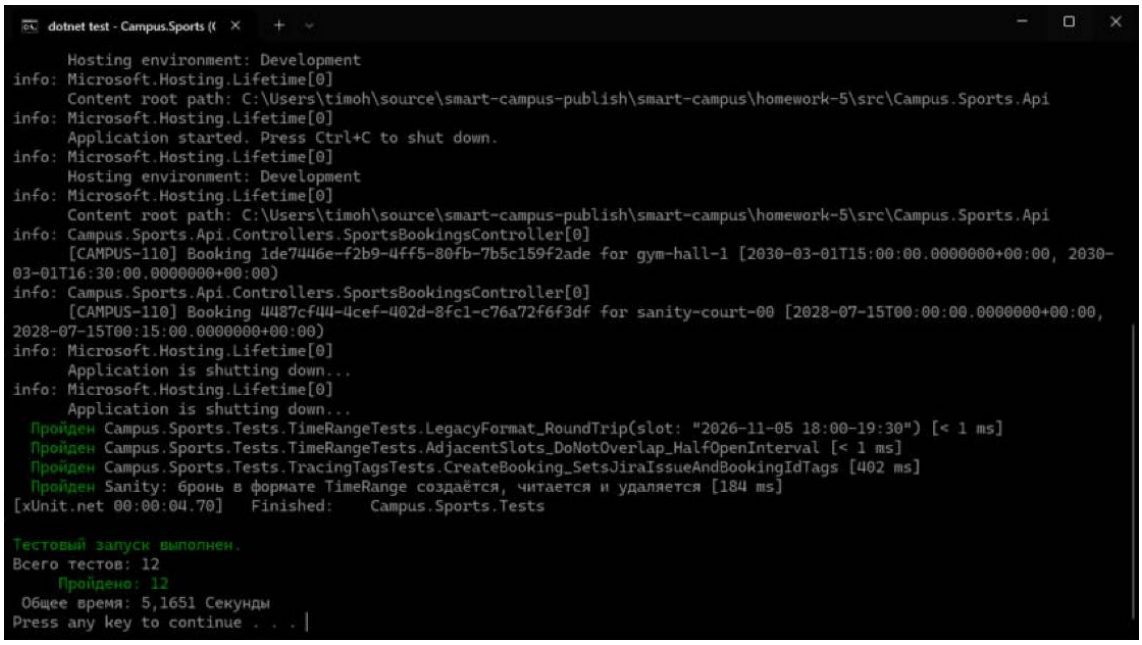


Рисунок 11 — Локальный запуск dotnet test: всего тестов 12, пройдено 12

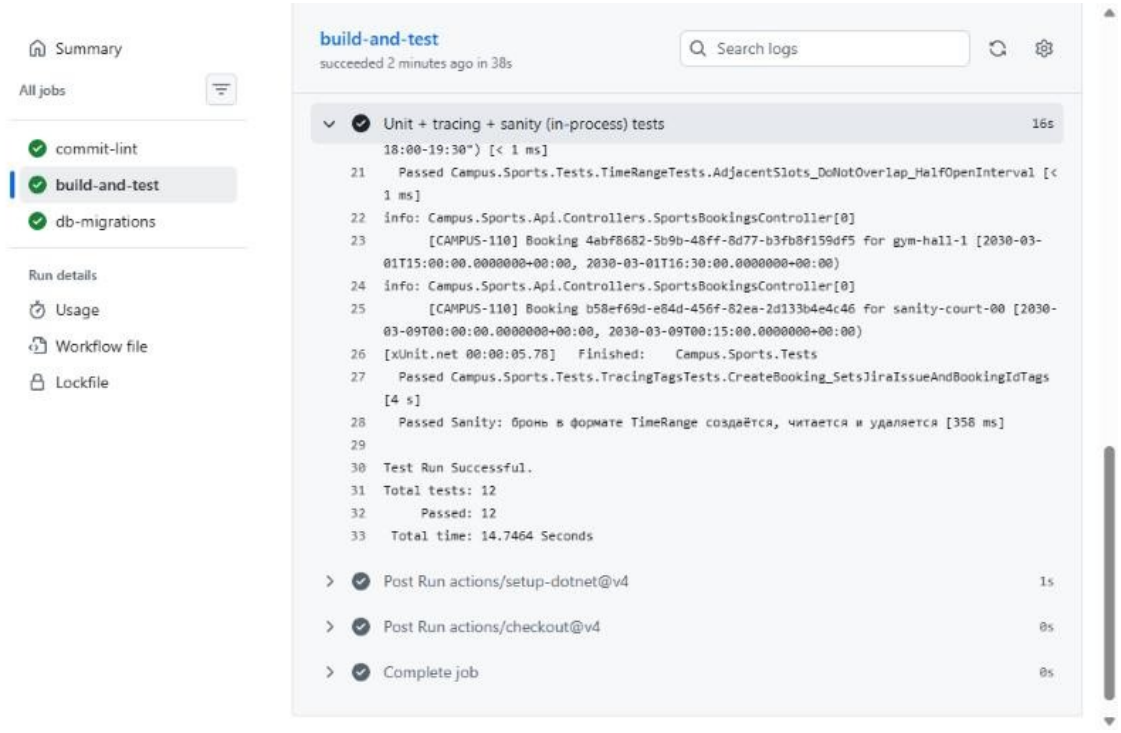
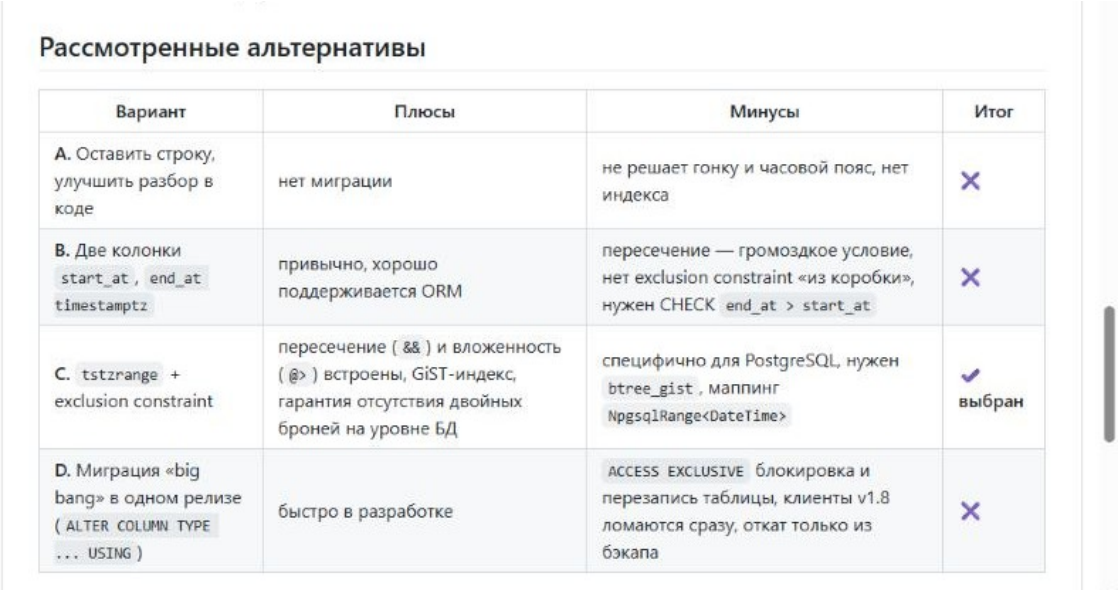


Рисунок 12 — Запуск тестов в GitHub Actions (job build-and-test)

7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ (ADR, CHANGELOG)

ADR-0009 (docs/adr/0009-timeslot-to-timerange.md) фиксирует архитектурное решение по варианту: контекст (двойные брони, часовые пояса, два формата строки, отсутствие индексов), принятое решение (TimeRange / tstzrange, exclusion constraint, миграция Expand and Contract в двух релизах, dual-write + триггер), рассмотренные альтернативы (рисунок 13) и последствия [9].



Рассмотренные альтернативы

Вариант	Плюсы	Минусы	Итог
A. Оставить строку, улучшить разбор в коде	нет миграции	не решает гонку и часовой пояс, нет индекса	✗
B. Две колонки `start_at`, `end_at` `timestamptz`	привычно, хорошо поддерживается ORM	пересечение — громоздкое условие, нет exclusion constraint «из коробки», нужен CHECK `end_at > start_at`	✗
C. `tstzrange` + exclusion constraint	пересечение (`&&`) и вложенность (`@>`) встроены, GiST-индекс, гарантия отсутствия двойных броней на уровне БД	специфично для PostgreSQL, нужен `btree_gist`, маппинг `NpgsqlRange<DateTime>`	✓ выбран
D. Миграция «big bang» в одном релизе (`ALTER COLUMN TYPE ... USING`)	быстро в разработке	`ACCESS EXCLUSIVE` блокировка и перезапись таблицы, клиенты v1.8 ломаются сразу, откат только из бэкапа	✗

Рисунок 13 — Раздел «Рассмотренные альтернативы» в ADR-0009

Отрицательные последствия: временное усложнение (две колонки, триггер, флаг) до v1.10.0 и точка невозврата при DROP COLUMN — поэтому фаза Contract вынесена в отдельный CR.

CHANGELOG.md ведётся по стандарту Keep a Changelog [10] и Semantic Versioning. Запись [1.9.0] - 2026-10-28 содержит разделы Added (TimeRange, колонка time_range, dual-write, теги OpenTelemetry, sanity-тест), Changed, Deprecated (поле timeSlot), Fixed (двойное бронирование, сдвиг времени на 3 часа, формат DD.MM.YYYY) и Breaking Changes (в 1.9.0 — нет, релиз обратно совместим). В разделе [Unreleased] анонсированы breaking changes версии 1.10.0 (рисунок 14).

1.9.0 - 2026-10-28

Added

- Тип `TimeRange` (интервал `[start, end)`) для броней площадок спорткомплекса; колонка `sports.bookings.time_range tstzrange` с CHECK-ограничением и GiST-индексом (CAMPUS-110).
- Поля `start` и `end` (ISO 8601 со смещением) в запросе и ответе `POST /api/sports/bookings`.
- Dual-write в `time_slot` и `time_range`, флаг `Migration:WriteLegacyTimeSlot` (CAMPUS-110).
- Триггер `trg_bookings_sync_time_slot` для согласованности колонок во время rolling update.
- Теги OpenTelemetry: `jira.issue`, `campus.booking_id`, `campus.facility_id`, `campus.time_range.start/end`, `campus.request.format`.
- Production Sanity-тест `ProductionSanityTests` с обязательной очисткой тестовой брони.
- Журнал ошибок миграции `sports.bookings_migration_errors`.

Changed

- Пересечение броней проверяется по интервалу `tstzrange &&` под `pg_advisory_xact_lock` по площадке.
- Время брони хранится в UTC, API отдаёт его со смещением `+03:00`.

Рисунок 14 — Запись о релизе 1.9.0 в CHANGELOG.md

ЗАКЛЮЧЕНИЕ

В ходе практической работы для модуля «Спортивный комплекс» ИС «Умный кампус» (вариант 10, задача CAMPUS-110) спроектирован безопасный релиз изменения формата TimeSlot → TimeRange.

Реализован единый цифровой след: каждый запрос к API несёт теги jira.issue и campus.booking_id, а цепочка «задача — коммит — CI — релиз — трейс — миграция» прослеживается от начала до конца. Оформлен Change Request на релиз v1.9.0 с обоснованным окном обслуживания (среда, 03:00–05:00), пошаговым планом работ и планом отката через kubectl rollout undo, а также описаны правила защиты ветки main с обязательными проверками CI.

Миграция базы данных выполнена по паттерну Expand and Contract в три фазы: добавление новой структуры с dual-write в коде, пакетный перенос исторических данных с контролем и удаление старой структуры отдельным релизом. Все скрипты проверены на PostgreSQL 16: миграция проходит без блокировок записи, идемпотентна, обнаруживает некорректные данные и откатывается. Итоговая схема физически исключает двойное бронирование площадок.

Написан Production Sanity-тест с обязательной очисткой тестовых данных, составлены ADR-0009 и CHANGELOG по стандарту Keep a Changelog. Работа позволила закрепить навыки управления изменениями, безопасной эволюции схемы БД без простоя и ведения документации по принципу Docs-as-Code.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OpenTelemetry. .NET — Instrumentation. – URL: https://opentelemetry.io/docs/languages/dotnet/instrumentation/ (дата обращения: 22.10.2026).
2. OpenTelemetry. Semantic Conventions — Attribute Naming. – URL: https://opentelemetry.io/docs/specs/semconv/general/naming/ (дата обращения: 22.10.2026).
3. GitHub Docs. About protected branches. – URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (дата обращения: 22.10.2026).
4. PostgreSQL 16 Documentation. ALTER TABLE. – URL: https://www.postgresql.org/docs/16/sql-altertable.html (дата обращения: 22.10.2026).
5. PostgreSQL 16 Documentation. Range Types. – URL: https://www.postgresql.org/docs/16/rangetypes.html (дата обращения: 22.10.2026).
6. Kubernetes Documentation. Deployments — Rolling Back a Deployment. – URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ (дата обращения: 22.10.2026).
7. ITIL 4. Change Enablement Practice. – URL: https://www.axelos.com/certifications/itil-service-management (дата обращения: 22.10.2026).
8. Sadalage P. Parallel Change (Expand and Contract). – URL: https://martinfowler.com/bliki/ParallelChange.html (дата обращения: 22.10.2026).
9. Nygard M. Documenting Architecture Decisions. – URL: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions (дата обращения: 22.10.2026).
10. Keep a Changelog 1.1.0. – URL: https://keepachangelog.com/ru/1.1.0/ (дата обращения: 22.10.2026).